

JAXB

Java Architecture for XML Binding

Marshalling · Unmarshalling · Schema Generation · Class Generation

What is JAXB?

A standard framework for binding Java classes to XML representations



Marshalling

Convert Java objects into XML — serialise your object graph to a file, string, or stream.



Unmarshalling

Convert XML back into Java objects — parse a document and recreate the object graph.



Schema Generation

Generate an XSD schema directly from annotated Java classes.



Class Generation

Generate annotated Java classes from an XSD schema using the xjc tool.

Core JAXB Annotations



Annotation	Purpose
<code>@XmlRootElement</code>	Marks a class as a top-level XML element
<code>@XmlAccessorType</code>	Controls which fields/methods are bound (e.g. FIELD, PROPERTY)
<code>@XmlAttribute</code>	Serialises a field as an XML attribute
<code>@XmlElement</code>	Serialises a field as a child XML element
<code>@XmlType</code>	Defines the XML type name and element ordering (propOrder)
<code>@XmlEnum</code>	Maps a Java enum to an XSD enumeration type
<code>@XmlEnumValue</code>	Specifies the XML string value for an enum constant

Marshalling: Java → XML

Annotated Class

```
@XmlRootElement(name = "library")
@XmlAccessorType(XmlAccessType.FIELD)
class Library {
    @XmlAttribute
    private String name;

    @XmlElement(name = "book")
    private List<Book> books;
}
```

marshal()

XML Output

```
<?xml version="1.0"
      encoding="UTF-8"?>
<library name="City Library">
  <book isbn="978-...">
    <title>Effective Java</title>
    <author>Joshua Bloch</author>
    <year>2018</year>
  </book>
</library>
```

Marshalling Code

```
JAXBContext ctx = JAXBContext.newInstance(Library.class);
Marshaller m = ctx.createMarshaller();
m.setProperty(JAXB_FORMATTED_OUTPUT, true);
m.marshal(library, new StringWriter());
```

Unmarshalling: XML → Java

XML Source

```
<library name="City Library">
  <book isbn="978-...">
    <title>Effective Java</title>
    <author>Joshua Bloch</author>
    <year>2018</year>
  </book>
</library>
```

unmarshal()

Java Object

```
Library lib = (Library)
    unmarshaller.unmarshal(source);

lib.getName();    // "City Library"
lib.getBooks();   // [Book{...}]
```

Unmarshalling Code

```
JAXBContext ctx = JAXBContext.newInstance(Library.class);
Unmarshaller u = ctx.createUnmarshaller();
Library lib = (Library) u.unmarshal(new StringReader(xml));
lib.getBooks().forEach(System.out::println);
```

Schema Generation: Classes → XSD

Annotations that enrich the schema

Annotation	XSD Effect
@XmlAttribute(required=true)	use="required"
@XmlElement(required=true)	minOccurs="1"
@XmlElement(defaultValue=...)	default="..."
@XmlType(propOrder={...})	Fixed element order
@XmlEnum + @XmlEnumValue	xs:enumeration

Generated XSD

```
<xs:complexType name="book">
  <xs:sequence>
    <xs:element name="title"
      type="xs:string"/>
    <xs:element name="year"
      type="xs:int"
      default="0"/>
  </xs:sequence>
  <xs:attribute name="isbn"
    type="xs:string" use="required"/>
</xs:complexType>
```

```
ctx.generateSchema(new SchemaOutputResolver() {
  public Result createOutput(String ns, String file) {
    StreamResult r = new StreamResult(new File(file));
    r.setSystemId(file); return r;
  }
});
```

Class Generation: XSD → Java

Using the xjc Compiler

```
# Basic usage
xjc library.xsd

# With output directory and package
xjc -d src/main/java -p com.example.library
library.xsd

# With custom bindings
xjc -d src/main/java -p com.example.library -b
bindings.xjb library.xsd
```

Flag	Purpose
-d <dir>	Output directory for sources
-p <pkg>	Java package name
-b <file>	Custom bindings (.xjb)
-verbose	Detailed output

Files generated by xjc

Library.java

Book.java

Genre.java

ObjectFactory.java

package-info.java

- @XmlRootElement class with List<Book>
- @XmlAccessorType with all mapped fields
- @XmlEnum with @XmlEnumValue constants
- @XmlRegistry factory for creating instances
- Namespace URI binding for the package

Maven Integration

Run xjc automatically at build time — no manual step required

pom.xml — jaxb-maven-plugin

```
<plugin>
  <groupId>org.glassfish.jaxb</groupId>
  <artifactId>jaxb-maven-plugin</artifactId>
  <version>4.0.5</version>
  <executions>
    <execution>
      <goals><goal>generate</goal></goals>
    </execution>
  </executions>
  <configuration>
    <schemaDirectory>
      src/main/resources/xsd
    </schemaDirectory>
    <generatePackage>
      com.example.library
    </generatePackage>
  </configuration>
</plugin>
```

Runtime Dependency (Jakarta EE / Java 11+)

```
<dependency>
  <groupId>com.sun.xml.bind</groupId>
  <artifactId>jaxb-impl</artifactId>
  <version>4.0.5</version>
</dependency>
```

Java 8 Note

JAXB is bundled with the Java 8 JDK — no extra dependency is needed.

Change imports from jakarta.xml.bind.* to javax.xml.bind.*

The Complete Round-Trip



Round-trip is lossless — repeat as many times as needed

During runtime:



Summary

- 1 Annotate** Use `@XmlRootElement`, `@XmlElement`, `@XmlAttribute` etc. on your model classes
- 2 Marshal** JAXBContext + Marshaller converts Java objects to XML in one call
- 3 Unmarshal** JAXBContext + Unmarshaller parses XML back into your Java objects
- 4 Generate Schema** `ctx.generateSchema()` produces an XSD from your annotated classes
- 5 Generate Classes** xjc (or the Maven plugin) builds annotated classes from any XSD

JAXB in JAX-RS Web Applications

Java 11 · Tomcat 9 · JAX-RS 2.1 · Jersey 2.35

Java 11 Impact

JAXB was removed from the JDK (JEP-320). It must be added as an explicit Maven dependency — Tomcat 9 does not bundle it either.

JAX-RS 2.1

Uses `javax.ws.rs.*` namespace. Jersey 2.29+ required for Java 11. Jersey auto-discovers JAXB message body readers and writers.

Auto Serialisation

Annotate your model with `@XmlElement`. Jersey marshals to XML or JSON automatically based on the client Accept header.

Key rule: use `javax.xml.bind` and `javax.ws.rs` namespaces (not `jakarta`) for the Tomcat 9 / JAX-RS 2.1 stack

Dependencies & Configuration



pom.xml — key dependencies

```
<!-- JAXB API (removed from JDK 11) -->
<dependency>
  <groupId>javax.xml.bind</groupId>
  <artifactId>jaxb-api</artifactId>
  <version>2.3.1</version>
</dependency>
<!-- JAXB Runtime implementation -->
<dependency>
  <groupId>org.glassfish.jaxb</groupId>
  <artifactId>jaxb-runtime</artifactId>
  <version>2.3.6</version>
</dependency>
```

Version compatibility

Component	Version
Java	11
Tomcat	9.x
JAX-RS	2.1
Jersey	2.35
JAXB API	2.3.1
JAXB Runtime	2.3.6

Resource Class & Auto Serialisation

1 Annotate the model

```
@XmlRootElement(name = "book")
public class Book {
    @XmlAttribute private String isbn;
    @XmlElement   private String title;
    @XmlElement   private String author;
    @XmlElement   private int    year;
    public Book() {} // required by JAXB
}
```

2 Write the JAX-RS resource

```
@Path("/books")
public class BookResource {
    @GET
    @Produces({APPLICATION_XML,
               APPLICATION_JSON})
    public List<Book> getAll() { ... }
    @POST
    @Consumes({APPLICATION_XML,
```

3 Jersey serialises automatically

Accept: application/xml → JAXB → XML

Accept: application/json → MOXy → JSON

Content-Type: */xml → JAXB unmarshal

Content-Type: */json → JSON unmarshal

4 JAXB exception mapper

```
@Provider
public class ErrMapper implements
    ExceptionMapper<NotFoundException> {
    public Response
```

Returning List<Thing> as XML

A bare List<Employee> has no XML root element — JAXB needs a wrapper class with @XmlRootElement

X Returns List directly — BREAKS

```
// Jersey throws:  
// MessageBodyWriter not found  
@GET  
@Produces(APPLICATION_XML)  
public List<Thing> getAll() {  
    return service.findAll();  
}
```

Returning List<Thing> as XML

A bare List<Thing> has no XML root element — JAXB needs a wrapper class with @XmlRootElement

✓ EmployeeList wrapper class

```
import javax.xml.bind.annotation.*;

@XmlRootElement(name="things")
public class ThingList {

    @XmlElement(name="thing")
    private List<Thing> things
        = new ArrayList<>();

    public ThingList() {}
    public ThingList(
        List<Employee> e) {
        this.things = e;
    }
}
```

✓ JAX-RS resource method

```
import javax.ws.rs.*;
import javax.ws.rs.core.*;

@Path("/things")
public class ThingResource {

    @GET
    @Produces(APPLICATION_XML)
    public Response getAll() {
        List<Thing> list = ...
        return Response
            .ok(new ThingList(list))
            .build();
    }
}
```